Setup and run guide

1. Extract and open the project

Extract the ZIP completely. Open the inner AI_Smart_Traffic_Signal_FYP folder containing requirements.txt, dashboard and models. Do not run from inside the ZIP.

Use Windows 10/11 with 64-bit Python 3.12, a browser and an internet connection for the initial package installation. VS Code is optional. A GPU, camera and separate SUMO installer are not required.

2. Create a Python environment

Open PowerShell in the project root:

```
py -3.12 --version
py -3.12 -m venv venv
.\venv\Scripts\python.exe -m pip install --upgrade pip
.\venv\Scripts\python.exe -m pip install -r requirements.txt
```

If py is unavailable, check python --version and use python -m venv venv with a Python 3.12 installation, or use its full executable path. Avoid the Microsoft Store alias if it does not launch Python. On the original PC, an existing working venv can be used; a copied ZIP needs its own environment. Packages are not bundled in the ZIP. Automatic SUMO discovery removes the need to set SUMO_HOME manually.

3. Start the simulation

From the same project root:

```
.\venv\Scripts\python.exe dashboard/app.py --port 5000
```

Open http://127.0.0.1:5000. Keep PowerShell open; press Ctrl+C there to stop the server. The default controller is trained DQN, using models/dqn_approach_best.zip when available. The supplied checkpoint requires no retraining.

Alternatively run .\run_simulation.ps1. The direct Python command above works if PowerShell blocks scripts. If port 5000 is occupied, add --port 5001 and open http://127.0.0.1:5001.

4. Use the dashboard

- Choose AI / trained DQN, density adaptive or fixed cycle from Controller.

- Pause/resume playback, change its speed, or reset the episode.

- Choose a road and vehicle type, then add a vehicle. Scheduled automatic traffic continues independently.

- Inject an ambulance or fire engine to observe emergency priority.

- Change Midnight/Daylight, junction/full-network view, or expand to fullscreen.

- Read queue, completed-trip wait, throughput and the saved benchmark table. Refresh with Ctrl+Shift+R after UI updates.

5. Check the installation

```
.\venv\Scripts\python.exe -m unittest discover -s tests -v
```

Ten integration tests start real SUMO instances. Run them in a separate terminal from the dashboard. Optional JavaScript syntax check, if Node.js is installed: node --check dashboard/static/app.js. Node.js is not needed to run the simulation.

6. Optional training and evaluation

Use a separate PowerShell terminal. Keep supplied model files backed up before retraining: these commands write new model outputs.

```
$env:TRAFFIC_USE_LIBSUMO = '1'
.\venv\Scripts\python.exe train.py --timesteps 100000 `
  --episode-seconds 1800 --randomize-routes `
  --out models/dqn_approach_v3.zip
.\venv\Scripts\python.exe eval/select_model.py --episodes 3 --episode-seconds 1800 --seed 9000
.\venv\Scripts\python.exe eval/evaluate.py `
  --model models/dqn_approach_best.zip --episodes 5 `
  --episode-seconds 1800 --seed 20000 --randomize-routes `
  --out eval/new_results.json
```

Validation chooses a checkpoint; evaluation compares it against both baselines. Seeds 10000-10004 were used for the supplied test results; use fresh seeds for new final tests. Restart the dashboard to load changed weights. To return that terminal to the default TraCI backend, run Remove-Item Env:TRAFFIC_USE_LIBSUMO, or open a fresh terminal.

7. Optional Colab and vision

Run .\venv\Scripts\python.exe prepare_colab.py. Open notebooks/Traffic_Training_Colab.ipynb with the VS Code Colab extension or in Colab. Connect a runtime, run cells in order, and upload notebooks/traffic_training.zip when prompted. Google runtime/Drive authorization uses your own account. Copy exported model artifacts into models, then restart the dashboard.

For the separate supplied-video detector demo:

```
Set-Location vision
..\venv\Scripts\python.exe detect_density.py `
  --source traffic_sample.mp4 --weights yolov8n.pt `
  --frame 60 --out demo_frame.jpg
```

The detector is optional and does not control the live SUMO simulation. Fine-tuning and mAP measurement require a labeled dataset.

Quick troubleshooting

- Missing modules: install requirements.txt using the exact venv Python command above, not a different system Python.

- SUMO missing: reinstall eclipse-sumo==1.27.1 in the same venv. Manual SUMO_HOME setup is normally unnecessary.

- Model incompatible: use the supplied four-approach best/v3 checkpoint. Legacy dqn_traffic models use a different state/action layout.

- Connection refused: keep the server terminal running, inspect its error, and use the same port in the browser.

- Old UI: perform a hard refresh. Stop and restart Flask after template changes.

- Congested traffic: compare adaptive control, reset, or inspect the original scenario generator. This does not mean the renderer is frozen.